Verifier-Side Silent Failures: A Taxonomy and a Negative-Control Test Method for Automated Verifiers in Agentic AI Governance

Author: Xinghua Zhao (Steven Zhao) · China **Publication status:** 已公开（2026-10-01 权利人授权推送） **© 权属宣告统一块（整体复制，不得删改）：**

```
© 2026 赵兴华 / Steven Zhao·China (ORCID 0009-0001-0512-1237). All rights reserved.
理论署名 (attribution) : LGD（Lifecycle Governance Doctrine / 全程治理论）— SynomosAI
initiative
名称状态 (name status)  : "SynomosAI" / "MedXpert" — 未申请实体注册、未申请商标注册
                          (not a registered legal entity; no trademark registered)
生产参考部署 (production reference, self-reported) : MedXpert
                    ← 非认证、非背书、非监管认可（not a certification or endorsement）
代码许可 (code license) : uibc-core = Apache-2.0 (see repo LICENSE)
                    本文本与理论表述不在 Apache-2.0 覆盖范围内
引用格式 (cite as)      : uibc-core/CITATION.cff · concept DOI 10.5281/zenodo.22821834
首次公开锚 (first public): 2026-09-17 13:31:45 UTC (commit cb6f11b)
                    外锚 (external anchor): Sigstore Rekor logIndex 2883389783
```

Abstract

Automated verifiers — policy gates, admission controllers, compliance checkers, CI quality gates, and agentic-AI governance validators — are increasingly granted the authority to decide whether an artifact, an action, or an agent is acceptable. The classical literature studies verifiers that fail loudly. This paper studies the opposite and less examined hazard: **verifier-side silent failures**, in which a verifier produces a *pass* although the condition it was meant to enforce no longer holds at the time of use. We (i) present a taxonomy of such failures, organized into five families and fourteen named modes spanning CI gates, test suites, data pipelines, and AI-agent harnesses; (ii) give a runnable, four-case negative-control test method that forces a verifier to demonstrate *discriminative power* — the ability to distinguish enforcing from non-enforcing implementations — rather than merely to execute; and (iii) show how the method is made tamper-evident by machine-readable specifications, a hardened expectation table, and a mutation matrix in the spirit of "who tests the tester?", all enforced by CI gates that mirror pre-commit hooks one-to-one. We ground the taxonomy in a fifty-year lineage — TOCTTOU race conditions, complete mediation, design by contract, runtime verification, mutation testing, the oracle problem, and fault-injection history checking — and argue that while each fragment is old, their aggregation into a first-class, verifier-side failure catalog for agentic-AI governance is new. A complete worked example (a use-time re-evaluation scenario) demonstrates all four cases, all four mutants, and a specification/code divergence check. We close with an honest boundary statement: reference-set completeness remains open (the oracle problem), general TOCTOU detection remains undecidable, and state-source trust remains a roadmap item.

Keywords: silent failure; negative control; verifier; agentic AI; CI gates; mutation testing; TOCTTOU; governance

---

1. Introduction

1.1 Motivation

Modern software and AI-agent pipelines rely on automated verifiers: linters and policy gates in CI, Kubernetes admission webhooks, document compliance checkers in regulated industries, and — with the rise of agentic AI — governance validators that decide whether an agent's planned action is permitted. When such a verifier crashes or raises an error, the failure is *loud* and can be handled (fail-shut or fail-open, as Kubernetes `failurePolicy: Fail|Ignore` makes explicit). But there is a quieter hazard: the verifier runs, reports success, and the enforced condition is **not actually enforced at the time of use**. We call these **verifier-side silent failures**.

A silent failure is worse than a loud failure precisely because it consumes trust: dashboards stay green, badges stay green, and the violation ships. The problem compounds in agentic-AI settings, where a verifier's "pass" is consumed by an autonomous actor that has no independent means of noticing that the verdict is stale.

1.2 Contributions

1. **A taxonomy and catalog** of verifier-side silent failures — five families, fourteen named modes, each with a reproduction, self-check, fix, and negative control (Section 3).
2. **A negative-control test method**: a runnable four-case protocol that requires a verifier to demonstrate discriminative power — catching an omitting implementation, a stale-read implementation, and an over-blocking implementation, while accepting a correct one (Section 4).
3. **A tamper-evidence layer**: machine-readable fixture specifications with startup spec/code divergence checks, a hardcoded shipped-behavior expectation table to prevent self-referential test circularity, and a four-mutant kill matrix operationalizing "who tests the tester?" (Section 4.3–4.4).
4. **A deployment discipline**: seven CI gates mirroring pre-commit hooks one-to-one, including a gate that runs the negative-control suite itself — so that breaking the tests' discriminative power turns CI red (Section 4.5).
5. **A worked case study** with full reproduction evidence (Section 5).

1.3 Honest scope statement

We do not claim to solve TOCTOU (it is provably undecidable in general [Bishop & Dilger 1996]), to complete the test oracle (an open problem [Barr et al. 2015]), or to establish trust in the state source a verifier consults (roadmap; Section 6). We claim that the *verifier-side* axis — the verifier itself silently passing — deserves first-class status in failure catalogs and test methods, and that the present negative-control method is an immediately deployable step in that direction.

2. Background and Related Work

Our 18-track literature review (6 tracks on negative controls × AI governance, 12 tracks on temporality, concurrency, meta-verification, and identity trust) found **no systematic counterpart** that treats verifier-side silent passing as a first-class, cataloged failure class in the agentic-AI governance context. Each ingredient, however, is well established:

Check-then-act gaps. TOCTTOU was systematized by Bishop & Dilger (1996), with lineage to RISOS (1976) and the PA project (1978); §5 of that work proves the general problem is not statically solvable. Engineering practice converged on atomic conditional writes (`UPDATE … WHERE version = ?`), HTTP ETag/If-Match precondition failures (412), and — conceptually — Saltzer & Schroeder's (1975) *complete mediation*: every access must be checked, and cached verdicts must be systematically invalidated when permissions change. Modern orchestrators keep re-learning the lesson: Kubernetes admission webhooks exhibit snapshot-isolation gaps (resourceVersion, subresource bypasses), and `failurePolicy: Ignore` is a *configured* fail-open switch — a structural source of silent failure shipped by default in production systems.

Use-time re-evaluation semantics. Design by Contract (Meyer 1992) and JML formalize permitting conditions as preconditions checked by runtime assertions; Runtime Verification (Havelund & Roşu) supplies monitor synthesis from specifications. Our method's permitting-condition polarity and use-time reads are hand-written instances of these ideas; monitor synthesis is a roadmap direction rather than a reimplementation.

Verifiers must be able to fail. Mutation testing (DeMillo, Lipton & Sayward 1978) makes explicit the *detection* stage: a test suite that cannot kill mutants is defective. Fault-injection history checking (Jepsen; deterministic simulation in FoundationDB and TigerBeetle) demonstrates that "all green" histories are exactly what needs to be caught. Our negative-control suite and mutant kill matrix instantiate the detection stage at the meta level — the object under test is the verifier, not the system.

The oracle problem. Barr et al. (2015) survey the difficulty of obtaining test oracles. Reference-set completeness in our method is therefore *explicitly qualified* as stipulated, not solved.

Freshness and causal order. Lamport (1978) and vector clocks establish that "use time" in distributed settings is a causal-order question, not a physical-clock question; our refusal to claim clock-based freshness semantics without a causal-order basis is deliberate.

State-source trust and identity. The IETF WIMSE working group's *AI Identity Management System* draft (draft-ietf-wimse-aims, September 2026) — successor to the individually submitted AI-agent authentication draft it adopted — together with SPIFFE/SPIRE forms a deployable identity stack for agents [11][14]; OWASP's *Top 10 for Agentic Applications 2026* (ASI02 "Tool Misuse & Exploitation", ASI03 "Identity & Privilege Abuse") and the beta *OWASP MCP Top 10* (MCP07 "Insufficient Authentication & Authorization") recognize authorization checking itself as an attack surface [12][13]. Our registry/evidence layer is positioned downstream of identity: it supplies *evidence*, not identity.

3. A Taxonomy of Verifier-Side Silent Failures

The published catalog (entries as of 2026-09-19, all `stable`) names **fourteen silent-failure modes in five families** [15]:

- **A · Vacuous Verification** — the check runs but has no discriminating power: *SF-001* zero items collected, exit 0; *SF-002* early return as an implicit skip; *SF-003* swallowed exceptions; *SF-004* sentinel values that make a crash indistinguishable from a bad result.
- **B · Uncounted Absence** — something required is missing, and missing does not count as failure: *SF-005* neutral markers logged but not counted; **SF-006 *undeclared means unchecked*** — the validator only checks what was declared; *SF-007* empty required values break nothing.
- **C · Wrong Evidence** — the signal used cannot prove the conclusion drawn from it: *SF-008* loading an artifact as evidence of use; *SF-009* exposure counted as usage; *SF-010* local green is not remote green.
- **D · Drifting Oracle** — the check itself degrades: **SF-011 *the always-green oracle*** — an exemption clause makes the check unconditional; *SF-012* misattributed failures.
- **E · Process & Environment** — the logic is correct; the environment lies to it: *SF-013* a stale copy contaminates the check; *SF-014* a hung process reports nothing and never dies.

Each entry ships a minimal reproduction, a self-check, a fix, and a negative control. Two families bite hardest at the meta level — including against the catalog's own tooling (Section 5):

- **SF-006 (undeclared means unchecked).** A verifier silently passes everything it was not declared to check. Coverage that was never written down is coverage that can regress invisibly.
- **SF-011 (the always-green oracle).** An exemption clause renders a check unconditional. A documented self-test that CI never executes is an instance of this shape: the verifier's discriminative power can regress while every status indicator — README, badges, dashboards — stays green.

The unifying shape across all fourteen modes: **a permit is issued on the strength of a check, but the check's validity does not survive to the moment of use**. Three recurring causes: (a) omission — the check exists in documentation or one branch only; (b) temporal drift — the check ran against a snapshot that has since changed (queue-time vs. use-time values); (c) polarity/coverage inversion — the check runs but its result is not bound to the decision (verifiers that block everything are caught by the same method, Section 4.3).

The shape is *infrastructure-independent in kind*: the catalog deliberately spans CI gates, test suites, data pipelines, and AI-agent harnesses, because the same failure mode recurs in each — every new technology stack pays the same tuition, which is why a shared method over a per-mode catalog is the right packaging.

---

4. The Negative-Control Test Method

4.1 Design goals

A verifier test must establish **discriminative power**, not merely execution: it must be *able to fail* in all the ways the verifier can be wrong. The method therefore requires four canonical cases (named here as in the published example):

Case	Purpose	Correct verifier	Omitting verifier	Stale-read verifier	Over-blocking verifier
A (condition omitted)	catch omission	FAIL (names the violated condition)	PASS (wrong)	PASS (wrong)	FAIL
B (temporal drift)	catch snapshot reads	FAIL	PASS (wrong)	PASS (wrong)	FAIL
C (clean baseline)	catch over-blocking	PASS	PASS	PASS	**FAIL (wrong)**
D (act-window lapse)	catch check-then-act gaps	FAIL	PASS (wrong)	PASS (wrong)	FAIL

A verifier "passes the method" only if it produces the *entire correct column*: fail A/B/D with a diagnosis naming the violated condition, and pass C. Case C is what catches the verifier that "fails everything" — loudly useless, silently score-friendly.

4.2 Permitting-condition polarity

All applicability conditions are expressed with uniform polarity via a single named flag (`no_applicable_freeze`: True = no freeze applies), so that mis-polarity is itself a detectable mutation rather than an ambiguity. FAIL diagnostics must **name the violated condition** (not merely exit non-zero) — attribution is part of the contract.

4.3 Tamper evidence: machine-readable spec, hardcoded expectations, mutants

1. **Fixture specification.** Each case's ground truth is a machine-readable JSON fixture; the simulation asserts spec/code agreement at startup. Injecting a bogus condition yields an immediate `SPEC/CODE DIVERGENCE` (exit 1) — the *test itself* is auditable, not just the verifier.
2. **Hardcoded expectation table.** Expected behaviors are fixed constants in the test, never derived from the implementation under test — preventing self-referential circularity ("the test passes because the code and the test were generated together").
3. **Mutation matrix.** Four programmatic mutants of the reference implementation — M1 (remove coverage differential), M2 (revert to snapshot read), M3 (remove act-window re-verification), M4 (polarity inversion) — must all be **KILLED**, each by the case(s) it deviates on: M1 by A, M2 by B, M3 by D, M4 by A/B/C/D (including C, catching over-blockers). A manual kill-basis review confirmed each expectation was derived from the *published* behavior table, not from running the mutated code (review record maintained in the repository's adversarial-review document [15]).

4.4 CI enforcement ("who tests the tester?", enforced)

The negative-control suite runs as its own CI job (on multiple Python versions), alongside gates for linting, catalog consistency, index links, leak scanning, integrity-manifest verification, and release consistency. Pre-commit hooks mirror the CI gates **one-to-one** (a documented parity rule: change one, change the other), so discriminative power cannot silently regress — the catalog's own SF-011 pattern is thereby applied to the catalog itself.

4.5 What the method does *not* do

It does not make the reference set complete (oracle problem, open); it does not trust the state source (the verifier is assumed to receive honest fixture data in the test; production state-source trust is a separate evidence-layer problem); it does not close TOCTOU (Case D demonstrates a deployable mitigation — re-verification at the action boundary — not a formal elimination).

---

5. Case Study: A Use-Time Re-Evaluation Scenario (T08-S5Q2)

Scenario: a governance verifier records at *enqueue* time that a control applies; by *use* time, conditions may have changed. Four implementations were compared: one that omits the check (A-detectable), one that reads a queue-time snapshot (B-detectable), one correct implementation that re-reads the permitting condition at the action boundary (the correct column), and one that blocks everything (C-detectable). The reference implementation and its negative-control suite, mutation matrix, and spec fixture were exercised end-to-end; the tamper check (bogus condition injection → startup divergence) was re-verified; and the full seven-gate CI run executed green across supported Python versions (run 36723523298; to be re-run at submission). The demonstration is reproducible from the public repository [15] with no randomness, no wall-clock dependence, and no external services — determinism was chosen deliberately so that any nondeterminism in a *deployment* of the method is itself a signal.

---

6. Discussion, Threats to Validity, and Roadmap

Threats. (1) Reference completeness: stipulated, per the oracle problem. (2) Single-scenario depth vs. breadth: the method is demonstrated deeply on one scenario; breadth is the catalog's per-profile roadmap. (3) The verifier's state source is assumed honest within the test; adversarial state sources are out of scope by construction and addressed via a separate evidence layer.

Roadmap. (a) Chaos injection: seeded fault injection into the fixture pipeline (Jepsen-style) so the verifier's own failure paths are exercised; (b) causal-order freshness semantics for distributed use-time re-evaluation (Lamport/vector-clock basis); (c) monitor synthesis from fixture specifications (RV) to replace per-case hand-written re-checks; (d) an evidence/attestation layer interoperating with AIMS/SPIFFE identity stacks — supplying *evidence* to, not competing with, identity infrastructure; (e) over-blocking metrics (pass-rate on clean corpora) to quantify the C-case property beyond a binary check.

Positioning. The catalog and method are governance-side complements to identity-stack standardization: identity says *who*; this work asks whether the *checks about what they may do* actually still hold when acted upon.

---

7. Conclusion

Silent failure on the verifier side is an old problem in new clothing, and every new technology stack — including agentic-AI governance — is currently paying the tuition again. The contribution here is not a new theory of races or oracles, but a packaging: a first-class taxonomy, a runnable negative-control method that forces verifiers to demonstrate discriminative power, tamper-evidence machinery that makes the tests themselves auditable, and a CI deployment discipline that makes "who tests the tester?" a green badge rather than a slogan. All artifacts are public, deterministic, and reproducible.

AI-assistance disclosure. This work was prepared with the assistance of AI research and coding agents (literature search, bibliographic verification, test implementation, and manuscript drafting) under the author's direction. The author reviewed, verified, and takes full responsibility for all claims, references, and code.

References

(All bibliographic data below verified first-hand against publisher records, DOI resolution, and canonical sources on 2026-10-01; corrections relative to the working draft are logged in the verification report.)

1. M. Bishop and M. Dilger. *Checking for Race Conditions in File Accesses*. Computing Systems 9(2):131–152, 1996. (No DOI; canonical: https://www.usenix.org/legacy/publications/compsystems/1996/spr_bishop.pdf)
2. J. H. Saltzer and M. D. Schroeder. *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9):1278–1308, 1975. DOI: 10.1109/PROC.1975.9939
3. E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo. *The Oracle Problem in Software Testing: A Survey*. IEEE Transactions on Software Engineering 41(5):507–525, 2015. DOI: 10.1109/TSE.2014.2372785
4. L. Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM 21(7):558–565, 1978. DOI: 10.1145/359545.359563
5. R. A. DeMillo, R. J. Lipton, F. G. Sayward. *Hints on Test Data Selection: Help for the Practicing Programmer*. Computer 11(4):34–41, 1978. DOI: 10.1109/C-M.1978.218136
6. B. Meyer. *Applying "Design by Contract."* Computer 25(10):40–51, 1992. DOI: 10.1109/2.161279
7. K. Havelund, G. Roşu. *Efficient Monitoring of Safety Properties*. International Journal on Software Tools for Technology Transfer 6(2):158–173, 2004. DOI: 10.1007/s10009-003-0117-6
8. J. Zhou et al. *FoundationDB: A Distributed Unbundled Transactional Key Value Store*. Proc. SIGMOD '21, pp. 2653–2666, 2021. DOI: 10.1145/3448016.3457559; TigerBeetle, *Simulation*

Testing for Liveness (VOPR), 2023, https://tigerbeetle.com/blog/2023-07-06-simulation-testing-for-liveness/ (online resource).
9. K. Kingsbury, P. Alvaro. *Elle: Inferring Isolation Anomalies from Experimental Observations*. PVLDB 14(3):268–280, 2020. DOI: 10.14778/3430915.3430918; Jepsen methodology, https://jepsen.io/ (online resource).
10. Kubernetes, *Dynamic Admission Control*, https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/ (accessed 2026-10-01): `failurePolicy` ∈ {Fail, Ignore}; `reinvocationPolicy` (mutating webhooks only) ∈ {Never, IfNeeded}.
11. IETF, *AI Identity Management System*, draft-ietf-wimse-aims-00, 2026-09-15 (work in progress), https://datatracker.ietf.org/doc/draft-ietf-wimse-aims/ — successor of draft-klrc-aiagent-auth (individual submission, 2026-03-02; taken into the WIMSE working group and later archived). IETF WIMSE WG charter, https://datatracker.ietf.org/wg/wimse/about/.
12. OWASP GenAI Security Project, *OWASP Top 10 for Agentic Applications 2026*, released 2025-12-09, https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/ (ASI02 "Tool Misuse & Exploitation"; ASI03 "Identity & Privilege Abuse").
13. OWASP, *OWASP MCP Top 10* (beta), MCP07 "Insufficient Authentication & Authorization", https://owasp.org/www-project-mcp-top-10/ (accessed 2026-10-01).
14. CNCF SPIFFE Project, *SPIFFE Specification* and *X.509-SVID*, https://github.com/spiffe/spiffe/tree/main/standards (accessed 2026-10-01).
15. The silent-failure catalog repository and its worked example (examples/T08-S5Q2), github.com/zhaoxinghua09-cell/silent-failure-catalog — fourteen modes, five families, seven CI gates; code MIT, prose all rights reserved with citation permitted. (Authorship/attributed form per final rights decision.)